
Software Requirements Specification

for Voting System

Version 1.0 approved

Prepared by Manan Chaturvedi, Andy Dang, Michael Dunn, Mark Tsai

Team 11

2/6/2025

Table of Contents

Table of Contents	ii
Revision History	iii
1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	1
1.5 References	1
2. Overall Description	1
2.1 Product Perspective	1
2.2 Product Functions	2
2.3 User Classes and Characteristics	2
2.4 Operating Environment	2
2.5 Design and Implementation Constraints	2
2.6 User Documentation	3
2.7 Assumptions and Dependencies	3
3. External Interface Requirements	3
3.1 User Interfaces	3
3.2 Hardware Interfaces	4
3.3 Software Interfaces	4
3.4 Communications Interfaces	4
4. System Features	5
4.1 Shuffle Flag Toggle	5
4.2 CSV File Open	5
4.3 Seat Number Prompt	6
4.4 Algorithm Prompt	7
4.5 CSV File Load	8
4.6 Ballot and Candidate Number	9
4.7 Droop Quota Amount	10
4.8 STV Audit File Prompt	11
4.9 STV Audit File Output	12
4.10 Plurality Algorithm	13
4.11 STV Algorithm	14
4.12 Display Election Results	16
5. Other Nonfunctional Requirements	16
5.1 Performance Requirements	16
5.2 Safety Requirements	16
5.3 Security Requirements	17
5.4 Software Quality Attributes	17
5.5 Business Rules	17
6. Other Requirements	17
Appendix A: Glossary	17

Appendix B: Analysis Models**18****Appendix C: To Be Determined List****19**

Revision History

Name	Date	Reason For Changes	Version
Manan	2/6/2025	Adding Introduction!	0.1
Andy	2/9/2025	Sections 1.1, 5.1, 5.2, 5.3 added (incomplete)	0.2
Mark, Michael	2/10/2025	Sections 4.1 to 4.6 and a use cases checklist	0.3
Mark, Manan, Andy	2/10/2025	Sections 4.7 to 4.10, 5.4, 5.5.	0.4
Mark	2/13/2025	Add priority, organizational benefit, and use case number in the trigger, as well as sections 1.2, 1.3, 1.4, 3.1, 3.2, and Appendix A (need to add more information in the future). Fix actors and section 5.3.	0.5
Mark	2/15/2025	Section 2.4, 3.3, 3.4, and 4.11, refine 1.2 and 3.1. Add Appendix A.	0.6
Mark, Manan, Andy	2/16/2025	Section 1.5, 4.12, 6, Appendix B and C. Modified 3.2.	0.7
Michael	2/16/2025	Sections 1-4, updating language and increasing clarity.	0.8
Mark	2/17/2025	Add software system into actors and update the use cases folder	0.9
Andy	2/17/2025	Sections 4.11 and 4.12	0.10
Manan	2/17/2025	Sections 2.1 - 2.3	0.11
Michael	2/18/2025	Sections 2.3, 2.5, 3.1, 4.1-4.5 refined	0.12
Mark, Michael, Andy	2/19/2025	Update section 1.1, Table of Contents accuracy	0.13
Manan, Michael, Mark, Andy	2/19/2025	Finalizing and revising documentation	1.0

1. Introduction

1.1 Purpose

This document outlines version 1.0 of the voting system's purpose, functionality, and development. This system assists in election processes using the single transferable vote (STV) or plurality voting methods. Our product is not a voting location; voters will not interact with it directly. Instead, when provided with ballot information, this system will determine the selected winners of the seats in elections fairly and transparently for the election officials.

1.2 Document Conventions

This document is using the IEEE System Requirement Specification (SRS) template. The priorities for higher-level requirements are supposed to be inherited by detailed requirements.

1.3 Intended Audience and Reading Suggestions

- For typical users who want to use this voting system to study the election process or use the voting system as a tool, we suggest reading Section 3 to understand its interfaces.
- For advanced or professional users who want to conduct elections with this voting system, we suggest reading the whole document to understand the system's interrelational parts in the context of one another.
- For programmers who want to fix bugs or improve the voting system, we suggest reading Section 4 to understand the use cases.

1.4 Product Scope

The voting system is a tool for processing ballots and determining winning candidates using a plurality of single transferable voting (STV) electoral system. Users can analyze or view the election results, output audit files, or conduct elections to fill candidate seats.

1.5 References

No other external documents are included in this documentation.

2. Overall Description

2.1 Product Perspective

This program will be a stand-alone product designed to define a voting system used by private or governmental agencies to calculate election results. It will include built-in testing capabilities to ensure legitimacy and fairness in election outcomes.

2.2 Product Functions

- Shuffle flag toggle: system testers can turn off ballot shuffling when launching the system (see UC_001).
- File open: The system prompts the user for a file name and opens the corresponding CSV file (see UC_002).
- Seat number prompt: The system prompts the user to input the number of seats to be filled by the system (see UC_003).
- Algorithm prompt: The system prompts the user to input which algorithm the system uses (see UC_004).
- CSV file load: The system loads data from the CSV file containing ballot information (see UC_005).
- Ballot and candidate number: The system counts the number of ballots and candidates using the ballot information entered by the user (see UC_006).
- Droop quota amount: The system calculates the Droop quota using the ballot information entered by the user (see UC_007).
- STV audit file prompt: The system prompts the user to enter a file name for the CSV file containing audit information (see UC_008).
- STV audit file output: After the system successfully performs an election, information containing election details is written to the user-defined output file (see UC_009).
- Plurality algorithm: The system calculates the election winner using the plurality algorithm (see UC_010).
- STV algorithm: The system calculates the election winner using the single transferable vote algorithm (see UC_011).
- Display election results: After the system successfully performs an election, election details are displayed on the user's screen (see UC_012).

2.3 User Classes and Characteristics

The two main user classes within the system are the election officials and the testers. The election officials will have control over which algorithm to use, the input of the ballot file's name, the number of seats, and, when applicable, the STV audit file. The tester ensures that the results will be consistent each time the program runs, regardless of the order in which the ballots are input. For this reason, only the tester has access to the "shuffle" flag, which will allow them to toggle between shuffled and unshuffled ballot input.

2.4 Operating Environment

The voting system will run in a CSE Labs environment, so its operating system is Ubuntu 24.04. Sections 3.2 and 3.3 list detailed requirements for the hardware and software interface.

2.5 Design and Implementation Constraints

The system must be written in C++ and run on University of Minnesota lab machines.

2.6 User Documentation

The information from the customer is proprietary. There are no other existing documentation components.

2.7 Assumptions and Dependencies

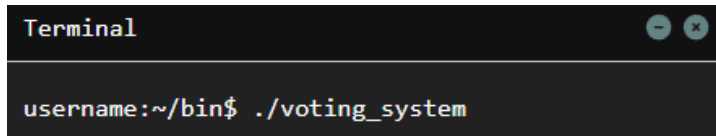
It is assumed that ballot information (e.g., number of candidates ranked, ranking order) entered into the system by the user is correct.

3. External Interface Requirements

3.1 User Interfaces

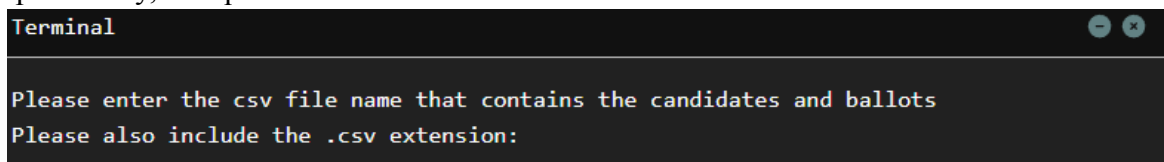
The only software component the user interface requires is a terminal. After the program starts with the command, it will provide instructions to guide the user through the election process. For instance, the system will first ask the user to input the name of the CSV file containing ballot information, and the user will input the file name before proceeding to the next step. If a user inputs an incorrect file name, the system will notify the user of its inability to locate the file and ask the user to re-input it. If the file name is valid, the system will inform the user that the file is successfully opened. Following is the visualized user interface of the example.

1. The program is started by typing “./voting_system” within the operating system’s command line interface.



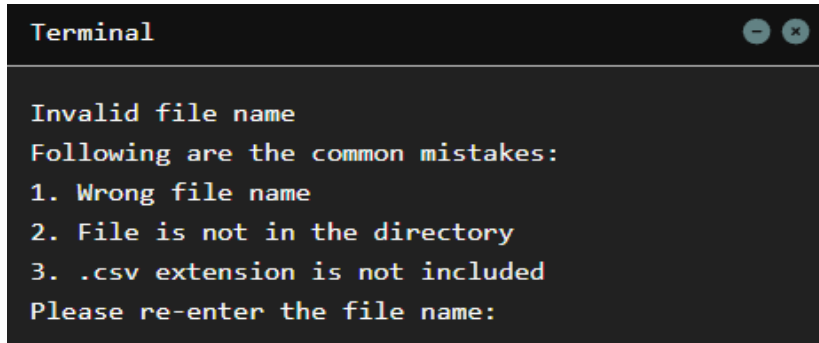
```
Terminal
username:~/bin$ ./voting_system
```

2. After the software starts, the system will first ask the user to input the name of the CSV file; specifically, it requires that the .csv extension be included.



```
Terminal
Please enter the csv file name that contains the candidates and ballots
Please also include the .csv extension:
```

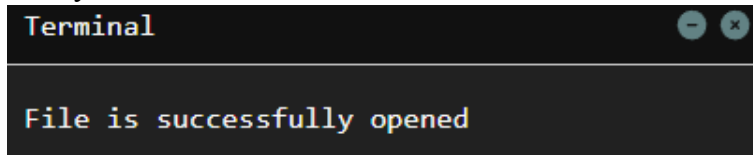
3. If the file name is invalid, the system prompts the user with an error message and possible causes of the error.



```
Terminal

Invalid file name
Following are the common mistakes:
1. Wrong file name
2. File is not in the directory
3. .csv extension is not included
Please re-enter the file name:
```

4. The system informs the user that the file name is valid and that the file is opened.



```
Terminal

File is successfully opened
```

3.2 Hardware Interfaces

This voting system must be able to run on the CSE Labs computers. Thus, the hardware requirements to run the system are as follows:

- CPU: 11th Gen Intel(R) Core(TM) i7-11700
- RAM: 32 GB

3.3 Software Interfaces

The voting system software components:

- Operating system: Ubuntu 24.04
- Programming Language and Build Tools: C++ with GCC compiler
- Libraries: C/C++ standard library

Data and messages input and output of the system:

- Input
 - CSV file name: The name of the CSV file to be read by the system.
 - CSV file: Contains the candidates and ballot information.
 - Number of seats: Determine how many seats will be filled in this election.
 - Type of algorithm: To perform the plurality or the STV voting.
 - STV audit file name: The name of the output STV audit file.
- Output
 - Display election details: Show the users the details and results of the election (see UC_012).
 - Output STV audit file: Save the STV voting process into a file that contains information about events that occur in a system for further inspection.

3.4 Communications Interfaces

There is no communication interface regarding this system.

4. System Features

4.1 Shuffle Flag Toggle

Name	Take the shuffle-off flag.
ID	UC_001
Description	When starting the software from the terminal, add the flag to indicate turning off the shuffle of votes into the voting algorithm.
Priority	Medium.
Actors	Software system and tester.
Organizational Benefits	It allows the software to be tested, ensuring the ballots are tallied fairly without error or bias.
Frequency of Use	Whenever the tester wants to test the software by turning the shuffle off.
Triggers	When the software starts from the terminal with the shuffle-off flag.
Preconditions	The shuffle-off flag is used.
Postconditions	The shuffle is turned off.
Main Course	1. The tester starts the software from the terminal with the shuffle-off flag. 2. The system stores the option of turning off the shuffle.
Alternative Course	There are no alternative courses.
Exceptions	There are no exceptions.

4.2 CSV File Open

Name	Open the CSV file.
ID	UC_002
Description	Prompt the user to input the CSV file name and open the file.
Priority	High.
Actors	Software system, election official, and tester.

Organizational Benefits	The use case allows the system to open the CSV file rather than having people manually enter data. By inputting the data automatically from the file, the system reduces labor costs and increases efficiency.
Frequency of Use	Every time the software is opened.
Triggers	When the software system is started.
Preconditions	There are no preconditions.
Postconditions	The CSV file is opened.
Main Course	1. The system prompts the user to enter the CSV file name. 2. The user enters the CSV file name. 3. The system opens the CSV file with the file name provided (see EX1, EX2).
Alternative Course	There are no alternative courses.
Exceptions	EX1 CSV file with the provided file name does not exist. 1. Prompt to acknowledge the user about an invalid CSV file name. 2. Return to the Main Course step 1. EX2 The open failed due to errors. 1. The system notifies the user that an error has occurred. 2. Exit the software.

4.3 Seat Number Prompt

Name	Prompt the user for the number of seats being elected.
ID	UC_003
Description	The user is prompted to enter how many seats will be filled by the election process.
Priority	High.
Actors	Software system, election official, and tester.
Organizational Benefits	This use case gives the system access to the number of seats to instantly perform later calculations for the droop quota, thus increasing efficiency.
Frequency of Use	Every time the software is opened.

Triggers	After UC_002 is finished, the file containing the votes is successfully read.
Preconditions	A valid filename is entered, a file in the directory with that filename is found, and the file is successfully opened.
Postconditions	The number of seats being elected is saved for later use in voting algorithm calculation.
Main Course	1. The system prompts the user to input the number of seats to be elected. 2. A positive integer is input by the user (see EX1). 3. The system saves the integer input by the user for later use.
Alternative Course	There are no alternative courses.
Exceptions	EX1: User inputs an invalid value. 1. The system notifies the user of an error. 2. Return to Main Course step 1.

4.4 Algorithm Prompt

Name	Prompt user for the algorithm to be used in the election process.
ID	UC_004
Description	The system prompts the user to enter which algorithm will be used to select the winners of the election's seats. The options for algorithms are plurality or STV.
Priority	High.
Actors	Software system, election official, and tester.
Organizational Benefits	Allows the organization to perform the election process it wants, either plurality voting or single transferable vote (STV), with no confusion or concern, thus providing clarity to the user to run the program faster with no errors.
Frequency of Use	Every time the software is opened.
Triggers	After UC_003 is finished, a valid number of seats to be elected is input.
Preconditions	A file name corresponding to a file in the directory is input and successfully opened. The user has input a valid (non-zero) integer for the number of seats

	that are being elected.
Postconditions	The user's algorithm selection is saved.
Main Course	1. The system prompts the user to input the algorithm for the election. The prompt includes the options for valid inputs, with '1' indicating STV and '2' indicating plurality. 2. An integer matching one of the prompted options is input by the user (see EX1). 3. The integer input by the user is converted to the corresponding algorithm and saved.
Alternative Course	There are no alternative courses.
Exceptions	EX1: User inputs an invalid value. 1. The system notifies the user of an error. 2. Return to Main Course step 1.

4.5 CSV File Load

Name	Load information in the opened CSV file to the data structure
ID	UC_005
Description	Read the opened CSV file, then process and store the content in the data structure.
Priority	High.
Actors	Software system.
Organizational Benefits	Allows the software system to perform an election based on ballot data in the file rather than through manual data entry, thus saving labor costs and increasing efficiency by having it read automatically.
Frequency of Use	Every time the software is opened.
Triggers	After UC_004 is finished, the user chooses a valid type of algorithm.
Preconditions	The CSV file is opened.
Postconditions	The CSV file is completely read to the end of the file, and a processed copy is stored in the data structure.
Main Course	1. Read and store the first line of the file, which contains the candidates (see EX1). 2. Read and store the remaining file, which contains all the

	ballots. For identification, assign each ballot a unique number (see EX1). 3. Close the file (EX2).
Alternative Course	There are no alternative courses.
Exceptions	EX1: The system fails to read the CSV file. 1. The system notifies the user that an error has occurred. 2. Exit the software. EX2: The system fails to close the CSV file. 1. The system notifies the user that an error has occurred. 2. Exit the software.

4.6 Ballot & Candidate Number Amount

Name	Determine the number of ballots and number of candidates.
ID	UC_006
Description	By iterating through the matrix of cast ballots, the number of ballots/votes and the number of candidates are counted and stored in separate variables.
Priority	High.
Actors	Software system.
Organizational Benefits	The software system performs this task by inspecting the data in the file rather than having people count the number of ballots and candidates. It saves labor costs and increases efficiency by finding and keeping that value automatically rather than searching for it.
Frequency of Use	Every time a CSV file containing cast ballots is successfully read.
Triggers	After UC_005 is finished, the CSV file is read to the end of the file, and a processed copy is stored in the data structure.
Preconditions	A valid filename is entered, a file in the directory with that filename is found, and the file is successfully opened.
Postconditions	The number of ballots cast is saved as a variable. The number of candidates is saved as a variable.
Main Course	1. The system iterates through the matrix containing cast ballots and counts each ballot containing a vote (see EX1, EX2). 2. The candidates in each ballot will be saved, ensuring no

	duplicate candidates exist. 3. The number of ballots is saved in a variable. 4. The number of candidates is saved in a variable.
Alternative Course	There are no alternative courses.
Exceptions	There are no exceptions.

4.7 Droop Quota Amount

Name	Determining the Droop quota.
ID	UC_007
Description	Calculating the Droop quota of an election simulated with an STV algorithm involves finding the total number of seats and ballots in the inputted CSV file.
Priority	High.
Actors	Software system.
Organizational Benefits	The software system can calculate the Droop quota based on the information obtained in the previous steps rather than having people do it, saving labor costs and increasing efficiency by determining the point where a candidate wins instantly by referring to this value.
Frequency of Use	Once after the number of ballots and seats are read from and ONLY if the algorithm is chosen to be STV. Otherwise, it is not run.
Triggers	The system receives a valid request for an STV algorithm, and the UC_006 is finished.
Preconditions	The user (election official or tester) inputs a request to use the STV algorithm. A valid number of seats and ballots exist.
Postconditions	A positive integer value of the Droop quota is saved to an integer variable.
Main Course	1. The system takes the number of valid seats and the number of candidates from their variable names. 2. Take the floor division of the numerator (number of ballots) by the denominator (number of seats plus 1) and then add 1 to that total value. 3. Save this positive integer value as the Droop quota.
Alternative Course	There are no alternative courses.

Exceptions	There are no exceptions.
-------------------	--------------------------

4.8 STV Audit File Prompt

Name	Prompting the STV audit report file name.
ID	UC_008
Description	The system requests that the user input a valid filename so it can create a text file to output the STV audit election log.
Priority	Low.
Actors	Software system, election official, and tester.
Organizational Benefits	Allowing the user to choose their output file name reduces confusion, the possibility of missing files, and the need for another repeat election with different results. It saves process time and maintains organizational credibility.
Frequency of Use	This case is performed only once when the algorithm chosen is STV.
Triggers	After UC_011 is finished, the STV algorithm is finished running its course with a valid result.
Preconditions	The system is finished running through the STV algorithm and has recorded the total steps of each round saved, along with the winners and losers, a valid number of seats, a valid number of candidates, and the different candidates running.
Postconditions	A text file containing the election winners, losers, total steps of each round, number of seats elected, and the different number of candidates.
Main Course	1. The system presents the user with an input box to ask for a file name. 2. The user provides a valid file name for the system to create (SEE EX1) 3. The system creates a text file. (SEE EX2)
Alternative Course	There are no alternative courses.
Exceptions	EX1: The user provides an invalid file name (e.g., the file name already exists within the directory) 1. The system sends an error message to the user. 2. The user is prompted again for a file name.

	EX2: The system fails to create a text file. 1. The system notifies the user that an error has occurred. 2. Exit the software.
--	---

4.9 STV Audit File Output

Name	Output STV audit report file.
ID	UC_009
Description	The system outputs a report that acts as an audit for the STV election.
Priority	Low.
Actors	Software system.
Organizational Benefits	To maintain organizational credibility, allow the users to inspect the election process and ensure it is performed correctly before submitting the results.
Frequency of Use	This case is performed only once after a successful read of the number of ballots and seats and ONLY if the algorithm is chosen as the STV algorithm. Otherwise, it is not run.
Triggers	After UC_008 is finished, the user inputs an STV audit report file name.
Preconditions	The user inputs a valid STV audit report file name, and the STV statistics are available.
Postconditions	An STV audit report file is output.
Main Course	1. Write election statistics into the file, including the ballots assigned to a candidate as the election progressed and pertinent election data (EX1). 2. Close the file (EX2).
Alternative Course	There are no alternative courses.
Exceptions	EX1: The system fails to write to the file. 1. The system notifies the user that an error has occurred. 2. Exit the software. EX2: The system fails to close the file. 1. The system notifies the user that an error has occurred. 2. Exit the software.

4.10 Plurality Algorithm

Name	Run the plurality algorithm
ID	UC_010
Description	Calculate the winner of the election using the plurality algorithm.
Priority	High.
Actors	Software system.
Organizational Benefits	Allowing users to run plurality elections enables the organization to perform different elections to cover possible usage, thus increasing efficiencies in running elections as needed and in multiple amounts.
Frequency of Use	This case is performed only once after a successful read of the number of ballots and seats and ONLY if the plurality algorithm is chosen. Otherwise, it is not run.
Triggers	The system receives a valid request for a plurality algorithm, and UC_006 is finished.
Preconditions	The user (election official or tester) inputs a request to use the STV algorithm. A valid number of seats and ballots are available.
Postconditions	The winner(s) of the plurality election are determined.
Main Course	1. The system sets the count of ballots received by each candidate to 0. 2. The system loops through each of the ballots. 3. The system increments the count of the candidate that the current ballot assigns itself to by 1. 4. The system determines the plurality election winner x times, where x is the number of seats to fill. It then determines the candidate who gets the most votes (see AC1). 5. The system finishes processing the election, with the winner(s) being the x amount of people who received the most votes, where x is the number of seats to fill.
Alternative Course	AC1 System determines that there are two or more people with the same amount of votes 1. The system flips a y-sided coin, where y is the number of people who have tied with the most votes. 2. The unchosen candidates will automatically be the winners of the next seat. 3. Return to Main Course step 5.

Exceptions	There are no exceptions.
-------------------	--------------------------

4.11 STV Algorithm

Name	Run STV algorithm
ID	UC_011
Description	Calculate the winner of a plurality election using the STV algorithm.
Priority	High.
Actors	Software system.
Organizational Benefits	The use case allows the users to run STV elections. It enables the organization to perform different elections to cover possible usage, thus increasing efficiency in running elections as needed and in multiple amounts.
Frequency of Use	This case is performed only once when the number of ballots and seats are read and ONLY if the algorithm chosen is plurality. Otherwise, it is not run.
Triggers	The system receives a valid request for a plurality algorithm, and UC_007 is finished.
Preconditions	The user (election official or tester) inputs a request to use the STV algorithm. A valid number of seats and ballots are available.
Postconditions	The winner(s) of the STV election are determined.
Main Course	1. The system does not detect the shuffle-off flag (see AC1). 2. The system shuffles the ballots. 3. The system calculates the Droop quota (see UC_007). 4. The system iterates through ballots, ordered by the first vote received, and distributes them. 5. The system checks the count that the current ballot holds (i.e., the first candidate that the ballot voted for) and determines if the corresponding candidate is on the losers list (see AC3). 6. After determining that the candidate is not on the losers list, the system distributes that ballot to the corresponding candidate (see AC6). 7. The system checks if candidates meet the Droop quota (see AC4). 8. Repeat Steps 5-7 for the next ballot until the distribution round is finished.

	9. After the distribution round finishes, the system eliminates the candidate with the fewest ballots and adds them to the top of the losers list (see AC2). 10. The system starts a new distribution round with the ballots of the most recently eliminated candidate, preserving their order. 11. Repeat steps 4-10 until all ballots have been processed.
Alternative Course	AC1 System detects the shuffle-off flag. 1. Return to Main Course step 3. AC2 There are multiple candidates with the fewest number of ballots. 1. The system checks when each candidate received their first ballot. 2. The system identifies the candidate that was last to receive their first ballot. 3. The system eliminates that candidate from the election. 4. Return to Main Course step 10. AC3 The system determines that the corresponding candidate is on the losers list. 1. The ballot increments its count (i.e., the next highest priority candidate). 2. Return to Main Course step 5. AC4 The system detects that a candidate has reached the Droop quota. 1. The system checks the elected list for open seats (see AC5). 2. When the system sees an open seat on the elected list, it adds the candidate to the end of the list. 3. The system removes every ballot distributed to the candidate from the ballot list. 4. Return to Main Course step 8. AC5 The system does not see an open seat on the elected list. 1. Return to Main Course step 8. AC6 The system detects that the current ballot will be the first ballot that the candidate receives. 1. The system records the order of the first ballot received for the candidate. 2. Return to Main Course step 7.
Exceptions	There are no exceptions.

4.12 Display Election Results

Name	Display Election Results
ID	UC_012
Description	Prints out the type of algorithm used in the election, the number of ballots, the number of seats filled, the number of candidates, and the winning candidates.
Priority	High
Actors	Software system.
Organizational Benefits	Allows election officials to see the results of their desired election, justifying the existence of this system and its results.
Frequency of Use	Once every time an election is processed.
Triggers	The election has been processed.
Preconditions	Either (UC_010) OR (UC_011) has been successfully executed.
Postconditions	The details (i.e., type of election, number of ballots, number of seats, number of candidates, winners, losers, percentage of votes received (plurality), order of winning and losing (STV)) of the current election are displayed on the screen.
Main Course	1. The system compiles and formats vital information about the current election. 2. The system prints out the election details on the screen.
Alternative Course	There are no alternative courses.
Exceptions	There are no exceptions.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The client requires the system to be able to process an election with up to 100,000 ballots. Moreover, it must process elections within 5 minutes.

5.2 Safety Requirements

There are no safety requirements regarding this system.

5.3 Security Requirements

The users interacting with his product will be election officials and testers who have been given specific access to this software. Therefore, the system has no additional security requirements.

5.4 Software Quality Attributes

This voting system provides election officials and testers with an easy-to-use interface and the functionalities needed to conduct an election while still delivering reliability, transparency, and correctness. The average user must train on the interface in less than 15 minutes. Testers can use the “shuffle-off” flag in the terminal to test the software, allowing them to view election results without shuffling ballots. At the same time, election officials can’t turn off the shuffle, ensuring a responsible election process within normal usage.

5.5 Business Rules

Election officials and testers run the same program as one another, each being able to input valid parameters such as the file name of the CSV used to run the election, the number of seats, the algorithm type, and - in the event of an election utilizing an STV algorithm - an output file name. However, only testers can toggle ballot shuffling off for testing purposes; election officials will have that on by default and are not permitted to change this feature.

6. Other Requirements

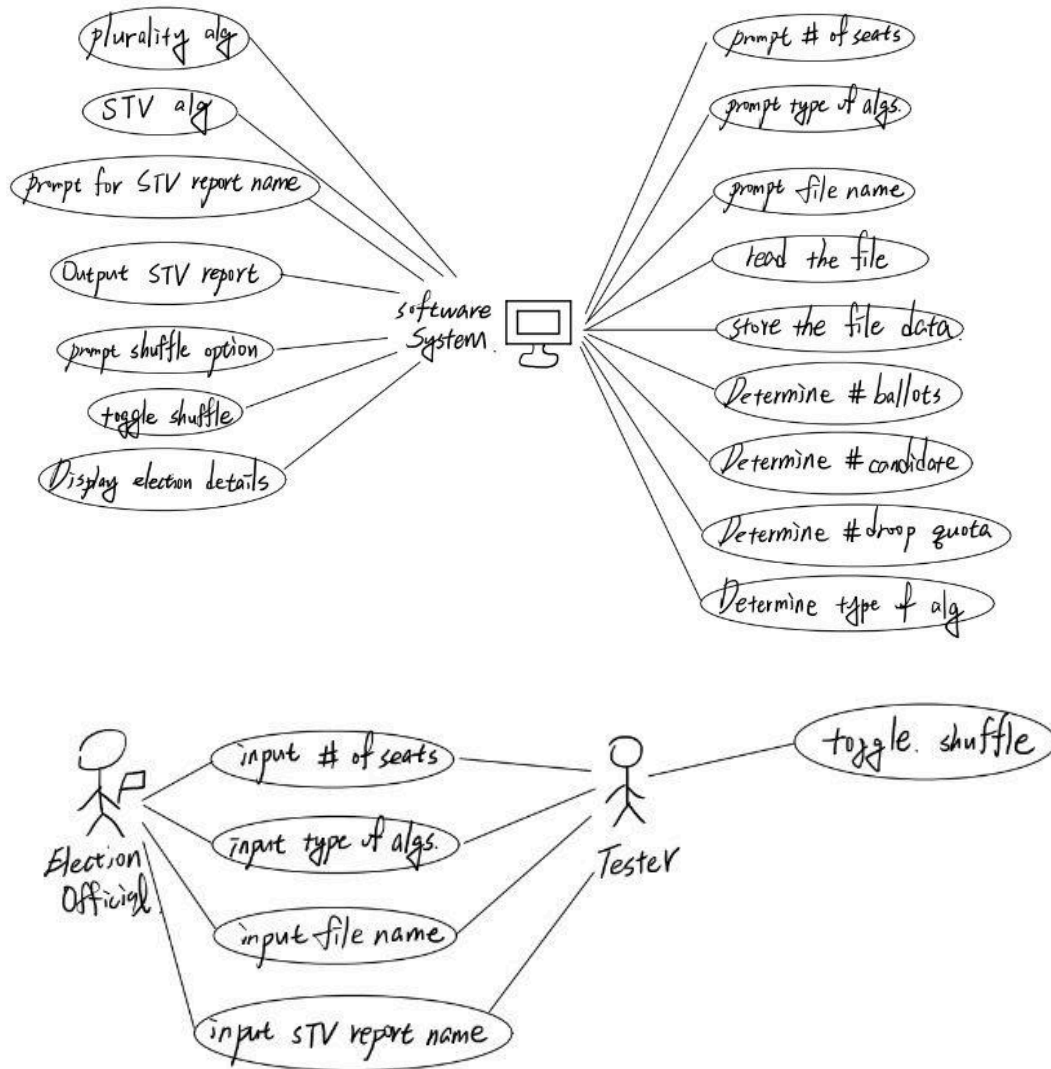
No other requirements are not covered elsewhere in the SRS. Not applicable.

Appendix A: Glossary

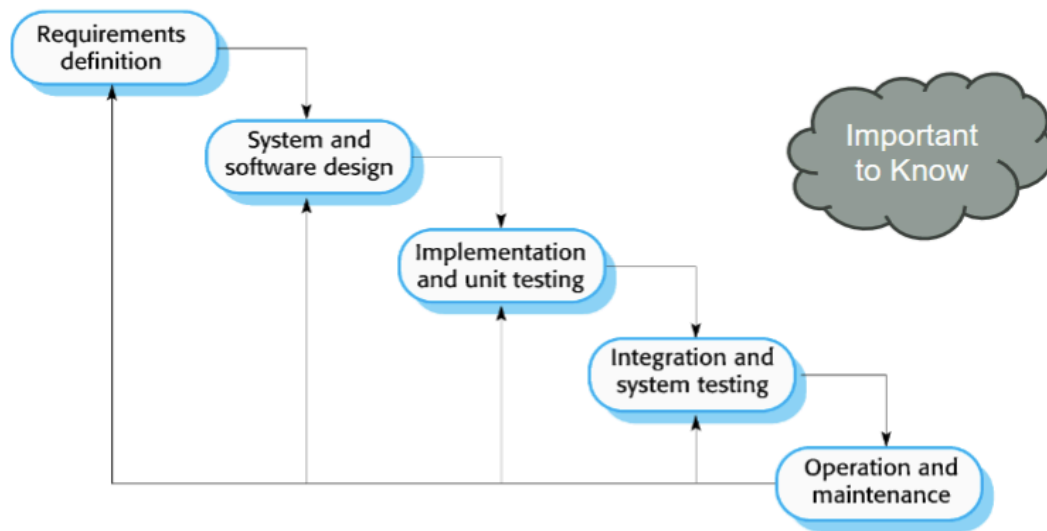
1. Linux: Linux is an open-source operating system (OS) that's used in many devices, including personal computers and supercomputers. Linus Torvalds created Linux in 1991.
2. C++: C++ is a general-purpose programming language developed to enhance the C language to include the object-oriented paradigm.
3. GCC: GCC (GNU Compiler Collection) is a collection of compilers and development tools supporting many programming languages.
4. Plurality: The number of votes cast for a candidate who receives more than any other but does not receive an absolute majority.
5. STV: Single transferable vote. A proportional representation voting method where voters rank candidates in order of preference. A candidate needs to reach a quota of votes to be elected. If a candidate exceeds the quota, their extra votes are transferred to voters' next preferences. If no one meets the quota, the candidate with the fewest votes is eliminated, and their votes are transferred. This process continues until all seats are filled.
6. CSV: Comma-separated values, a format for saving data in a text file. CSV files are commonly used to store tabular data, such as numbers and text, in a structured format
7. Flag: A flag is a variable that stores a binary value to indicate a condition or event. Flags are used to control program flow, trigger actions, and maintain program state.

Appendix B: Analysis Models

Use Cases and Actors Diagram



Waterfall Software Development Lifecycle (SDLC) Diagram



Appendix C: To Be Determined List

Nothing is TBD. Not applicable.